

Key-Value Caching in Transformer Models (Revised and Expanded)

Transformers generate text by attending to all previously seen tokens for each new token. Naïvely, this means redoing the same attention computations for every token, which is very wasteful. **Key-value (KV) caching** solves this by *saving* the computed key (K) and value (V) vectors from past tokens, so that the model can *reuse* them for future tokens. In practice, this turns a costly per-token computation into a quick lookup-and-append operation, greatly speeding up generation ¹ ².

Imagine writing a long essay and each time re-reading from the beginning – that’s standard inference. KV caching is like leaving bookmarks or notes on important paragraphs so you don’t have to re-read everything. At each new token, you only process the new token’s query (Q) against the saved K/V of all past tokens. This is illustrated in Figure 1: at token 1 we compute and store (K1,V1); at token 2 we load (K1,V1) and append (K2,V2); and so on ³. The attention score for a new query is then computed as $Q \cdot K^T$ using the cached keys, rather than recomputing all past keys.

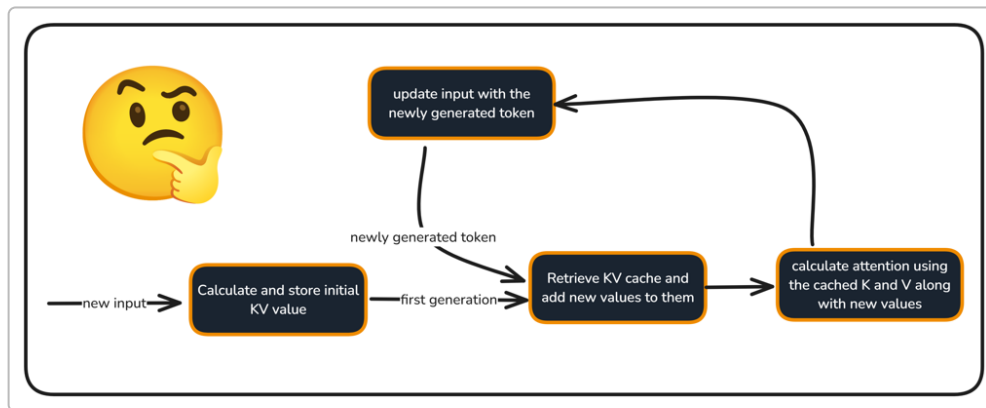


Figure: KV-caching workflow. Keys and values (K, V) from the initial input tokens are stored in cache. For each new token, the model retrieves the cache, appends the new K/V, and computes attention using the combined K/V (from cache and the new token) ³.

- **Step-by-step Process:** Key steps in KV caching are:
- **First token (initial input):** Compute its key and value vectors and store them in cache.
- **Each new token:** Compute its query (Q). Retrieve the stored keys/values (K, V) from cache, concatenate the new token’s K and V, and compute attention ($\text{softmax}(Q \cdot K^T) \cdot V$).
- **Update cache:** Append the new K and V to the cache. Repeat until generation is complete ³.
- **Reuse without recompute:** This avoids repeating K/V computations, so generation stays fast even as text grows.

- **Benefits:** Compared to standard generation, KV caching trades extra memory to store K/V for *less computation*. Standard inference “repeats the same calculations for every word,” but with KV cache “the model reuses past calculations for faster results” ⁴. As shown by benchmarks, turning on `use_cache=True` typically yields large speedups with *no accuracy loss* ⁵ ⁶. For example, one GPT-2 example saw generation go from ~61s down to ~11.7s ($\approx 5.2\times$ faster) when using caching ⁵.
- **Analogies:** KV caching is akin to using a **scrapbook** or **notepad** while thinking – once you’ve done a sub-calculation, you jot it down. Then future work can refer to your notes instead of re-deriving it. Another analogy (used in vLLM’s design) is **virtual memory**: we break memory into fixed-size “pages” that can be shared and moved around, letting us treat a fragmented cache like one large address space ⁷.

Cache Structure and Memory Growth

Each cached entry stores the K/V vectors for one token across all layers. Concretely, for a decoder-only model with L layers, H heads, head dimension d , and precision size, the **per-token cache size** is:

$$2 * L * (H * d) * \text{precision_in_bytes}$$

The factor 2 is for both K and V ⁶. For example, a 7B-parameter Llama-2 model (hidden size 4096, presumably 32 heads $\times$ 128-dim) in FP16 uses about 2 GB of cache per 4096-token sequence (batch size 1) ⁸. In general, KV cache memory grows linearly with *batch size* and *sequence length* ⁶ ⁸, which is why efficient caching is essential to scalability. If unbounded, large contexts simply exhaust GPU RAM.

- **Dynamic vs Static Cache:** Most frameworks use a *dynamic cache* that grows as needed. For example, Hugging Face’s `DynamicCache` (the default) allocates more storage on each new token. A *static cache*, by contrast, pre-allocates a large fixed-size buffer. Static caches enable compiler optimizations (e.g. easier kernel fusion) because they mask all unused slots, but they often waste computation on padding ⁹.
- **Offloading:** To save GPU memory, some libraries support **cache offloading**. Here, only the current layer’s cache is kept on GPU; completed layers’ K/V are moved to CPU RAM (and prefetched when needed) ¹⁰. This frees GPU space but incurs latency and PCIe costs. Practically, you enable offloading (in Hugging Face Transformers) by specifying `cache_implementation="offloaded"` (for dynamic cache) or `"offloaded_static"` (for static) ¹¹. Offloading is useful to avoid out-of-memory on small GPUs, at the cost of some throughput ¹⁰.
- **Memory vs Speed Trade-offs:** There is an inherent trade-off: a larger, full-precision cache means less recomputation (fast but memory-heavy), while smaller or lower-precision caches save RAM at the cost of extra work. For example, an 8-bit (FP8) cache can be 4 $\times$ smaller than 32-bit, but each step requires dequantization overhead ¹². Multi-query or grouped-query attention is an architectural way to reduce cache size by sharing keys among heads (see below). Ultimately, practitioners must **profile** their workloads. If GPU RAM is abundant, use full precision and largest possible cache for speed. If memory is tight, enable offloading or 8-bit caching ¹² ¹⁰.

Chapter 2: Framework-Level Implementations

Different frameworks implement KV caching with various optimizations. Here we cover key approaches in popular libraries.

2.1 Hugging Face Transformers

Built-in Caches: By default, Hugging Face models use `use_cache=True`, which automatically returns `past_key_values` in each forward pass. This is the **dynamic cache**: as you call `model.generate(...)`, the library internally saves K/V at each step for you ⁵ ². You can also construct cache objects manually: Hugging Face provides classes like `DynamicCache`, `StaticCache`, `QuantizedCache`, and their offloaded variants.

- **DynamicCache:** Grows per token. Supports sliding-window models (the cache stops growing at the window size) and offloading.
- **StaticCache:** Pre-allocates a full-size cache (e.g. equal to max context length). Enables static graph optimizations (via `torch.compile` or JIT) because tensor sizes are fixed, but at a large memory cost ⁹.
- **QuantizedCache:** Stores K/V in low precision (e.g. 4-bit via BitsAndBytes or custom schemes). E.g., `cache = QuantizedCache(...)`. This can shrink KV memory 4–8× ¹² but may degrade accuracy if too aggressive. In practice, 8-bit (FP8/E4M3) is popular, since 4-bit or 2-bit often hurt quality ¹². (Note: FlashAttention kernels typically don't support quantized cache, so Transformers may switch to a non-flash backend when using quantized KV.)
- **Offloaded Cache:** As noted, both Dynamic and Static caches can be *offloaded*. You enable it by either setting `cache_implementation="offloaded"` in `generate()`, or by constructing the cache with `offloading=True` ¹¹.

Code Example: Below is simple code using the Hugging Face API. It demonstrates default KV caching and a manual prefix cache.

```
from transformers import AutoTokenizer, AutoModelForCausalLM, DynamicCache,
StaticCache
import torch, copy

model_id = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id).to("cuda")

# 1. Basic generation with automatic caching
prompt = "Once upon a time"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=50, use_cache=True)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

```

# 2. Prefix caching (static cache)
prefix = "You are a helpful assistant. "
static_cache = StaticCache(config=model.config, max_cache_len=100)
inputs = tokenizer(prefix, return_tensors="pt").to(model.device)
# Prefill cache without gradient
with torch.no_grad():
    cache = model(**inputs, past_key_values=static_cache).past_key_values

# Now use this cache for multiple prompts sharing the prefix
questions = ["How are you?", "Tell me a joke about cats."]
for q in questions:
    text = prefix + q
    inputs2 = tokenizer(text, return_tensors="pt").to(model.device)
    pkv = copy.deepcopy(cache) # copy the prefix cache
    outs = model.generate(**inputs2, past_key_values=pk, max_new_tokens=30)
    ans = tokenizer.decode(outs[0], skip_special_tokens=True)
    print(ans)

```

```

# 3. Chat loop with DynamicCache
conversation = [
    {"role": "user", "content": "Hello, who are you?"},
    {"role": "assistant", "content": "I am an AI assistant."}
]
chat_cache = DynamicCache(config=model.config)
for turn in conversation:
    text = tokenizer.apply_chat_template([turn], add_generation_prompt=True)
    inputs = tokenizer(text, return_tensors="pt").to(model.device)
    out = model.generate(**inputs, max_new_tokens=50,
    past_key_values=chat_cache)
    reply = tokenizer.decode(out[0, -50:], skip_special_tokens=True)
    print(reply)

```

These examples use `past_key_values` (via `use_cache=True` or passing a cache object). In the chat loop, the `DynamicCache` grows with each turn, so each reply attends to the full history without re-encoding it. This is how multi-turn conversations are handled efficiently.

2.2 vLLM (PagedAttention)

[vLLM](#) is a high-performance LLM serving engine that introduced **PagedAttention**. The core idea is to treat the KV cache like an OS does virtual memory ⁷. Instead of one giant contiguous cache per request, vLLM splits each sequence's cache into **fixed-size blocks** (pages) of tokens. A lightweight "block table" (analogous to a page table) maps these logical blocks to physical memory chunks. Blocks can be placed anywhere in GPU memory (eliminating contiguous slab allocation) and even shared between sequences that share prefixes ⁷.

This design has big benefits:

- **No fragmentation:** Each new block is allocated only when needed, so there's no large empty reserve. If you have 1000 tokens, vLLM allocates exactly $\text{ceil}(1000/\text{block_size})$ blocks, no more ¹³. Internal and external fragmentation are almost eliminated (wasted space is at most $\text{block_size}-1$ tokens per sequence ¹⁴).
- **Shared prefixes:** When many sequences share a prefix (e.g. in beam search or multi-user settings), they point to the same physical blocks. Only on divergence does vLLM make a copy of the block (copy-on-write). Copying one block is far cheaper than copying thousands of token-states ¹⁵. In practice, this can reduce KV memory by ~50% when reusing long prefixes ¹⁵.
- **Flexible eviction/swapping:** Since KV is in blocks, vLLM can evict or swap out individual blocks if GPU memory fills up ¹⁶. It can even drop a block and *recompute* it later if needed, which can be faster than slow PCIe swaps for small blocks.

The *PagedAttention kernel* loops over each block: loading its keys, scoring them against Q, then loading its values, and accumulating the result ¹⁷. Mathematically this is identical to standard attention, but avoids large monolithic memory operations. As a result, vLLM achieves **near-zero memory waste and ~2-4× throughput gains** over naive caching (especially under high concurrency) ¹⁵ ¹⁸. In fact, benchmarks show vLLM easily doubles or quadruples tokens/sec versus the standard Transformers pipeline when batching requests (by packing more sequences into GPU memory) ¹⁵ ¹⁸.

Using vLLM: The API is simple. For example:

```
from vllm import LLM

# Create an LLM service with tensor parallelism and large batch capacity
llm = LLM(model="facebook/opt-6.7b", tensor_parallel_size=4,
max_num_batched_tokens=8192)
# Generate for multiple prompts in one call
results = llm.generate(["First prompt...", "Second prompt..."])
print(results)
```

The vLLM engine handles all paging, caching, and batching internally. It also supports dynamic batching, so you can keep issuing prompt lists and it will optimally group them.

Analogy: Think of vLLM's cache as a **bookshelf** with many small books (blocks) instead of one giant encyclopedia volume. Each book has a fixed number of pages (tokens). Two readers (sequences) reading the same chapter (prefix) will simply open the same book; only when they diverge does one reader take a new copy. This yields excellent memory sharing.

2.3 FlashAttention

[FlashAttention](#) is a specialized GPU kernel that implements exact softmax attention *fusion*. Its goal is to **minimize memory movement** by fusing the QK and softmaxV computations. In other words, it loads and processes a "tile" of the QK matrix, writes out the result, then repeats — rather than writing intermediate results to global memory each time ¹⁹.

In practice, FlashAttention can **speed up the attention math by 2–3×** on modern GPUs (especially A100/H100) compared to a naïve implementation. As NVIDIA notes: *“FlashAttention improves performance by minimizing memory movement costs.”* ¹⁹ . This is independent of KV caching: whether you have cache or not, the matrix multiplications (Q·K) and softmax will be faster.

However, there is a caveat for autoregressive (incremental) generation: FlashAttention assumes full matrices. Using it incrementally requires padding the new query vector so its position aligns with the existing keys, which can waste some work. Some frameworks handle this automatically under the hood. The benefit is usually worth it: in a bottleneck where QK is dominating time, FlashAttention reduces per-step latency.

Analogy: FlashAttention is like re-arranging assembly-line steps to avoid moving parts back and forth. By computing small blocks completely before moving on, it reduces trips to memory.

2.4 NVIDIA TensorRT-LLM

[TensorRT-LLM](#) is NVIDIA's high-performance inference library for LLMs. It incorporates many of the ideas above and adds its own features:

- **Paged KV Cache:** Like vLLM, TensorRT-LLM supports a paged (block-based) cache layout. Under the hood it uses the same “paged attention” strategy to avoid fragmentation.
- **Quantized Cache:** It allows 8-bit (FP8 or INT8) KV caches for memory savings.
- **Circular Buffer:** An optional circular (ring-buffer) KV, which recycles old entries in a fixed window (useful if you're sure you only need the last N tokens).
- **Cache Reuse (Sharing):** TensorRT-LLM can share and reuse cache pages across requests with identical prefixes. It even has advanced **priority-based eviction**: you can assign “priority scores” to certain token ranges. High-priority ranges (e.g. system prompts) will stay in cache longer, while low-priority ones can be evicted first. NVIDIA reports that this fine-grained control *“improves cache hit rates by around 20%”* ²⁰ .
- **KV Cache Events:** It provides an API to hook into cache updates, enabling external schedulers or routers to make KV-aware decisions (useful in large-scale serving).

Overall, TensorRT-LLM lets you tailor the cache policy to your workload. By default it uses LRU, but these new features let an engineer say “keep these key tokens” for faster warm-starts, etc.

Code: Usage is via the `trtllm-serve` or Triton server APIs, but conceptually it's similar to HF/vLLM: you send prompts and get tokens. Configuration is done via their builder flags (e.g. `--use_paged_context_fmha enable`, `enableBlockReuse=true`, etc) and RTX-specific API calls.

Chapter 3: Use Cases and Patterns

3.1 Optimizing Latency and Throughput

KV caching is primarily about speed. By avoiding re-computation, **per-token latency** drops dramatically. Moreover, when handling many simultaneous requests (high throughput), caching often unlocks far better GPU utilization. NVIDIA observes that during generation GPUs tend to be **memory-bound**: simply batching

more queries with caching packs more work into the GPU's fast cores. For example, benchmarks show vLLM achieves **2–4× higher tokens/sec** than the standard pipeline at various batch sizes ¹⁵ ¹⁸ .

Best practices:

- **Batch Requests:** Group multiple prompts into one batch. Even though each prompt has its own independent KV cache, batching lets the GPU perform one large matrix multiply for all, improving arithmetic intensity. Both vLLM and TensorRT support dynamic batching of streaming requests, so gather what you can.
- **Use Fast Kernels:** Wherever possible, use optimized attention kernels like FlashAttention or Triton fused kernels. These don't change cache size, but they speed up the attention math itself ¹⁹ .
- **Compile/Optimize:** With static shapes (e.g. using `StaticCache` or `torch.compile`), JITted code can eliminate branches and maximize throughput (some reports show ~4× speedups with compilation).
- **Large Context Windows:** If you need long inputs, use frameworks that support them natively. vLLM and TensorRT handle thousands of tokens gracefully. In HF Transformers, you may need a `StaticCache` of the full size to allow JIT – otherwise dynamic resizing can incur overhead.

If your use case is low-latency (like a chatbot), make sure to optimize **inter-token time**. For example, vLLM's "chunked prefill" mode interleaves generation with background work to reduce time-to-next-token.

In general, *turning on caching is free speed*. Hugging Face notes that simply setting `use_cache=True` in `model.generate()` can speed things up by 5× or more, with no downsides ⁵ . Always enable it for generation.

3.2 Cache Management and Policies

In real systems, one often layers strategies around the cache:

- **Offloading/Multi-GPU:** If GPU memory is a constraint, offload parts of the cache to CPU or distribute it across GPUs. For instance, increase tensor- or pipeline-parallelism so each GPU holds only some layers' cache. These methods trade throughput for memory savings (e.g. offloading adds PCIe overhead ¹⁰). Use them if you hit OOM.
- **Prefetching/Disaggregation:** Some systems (e.g. NVIDIA Triton) allow pre-filling prompts on CPU or other nodes, then sending only needed cache pages to GPUs. This can hide latency and balance load.
- **Eviction Policies:** By default, unused cache grows until memory runs out, then whole requests fail. Advanced systems let you control eviction. As mentioned, TensorRT-LLM's priority eviction lets you pin important tokens in cache ²⁰ . In custom servers (e.g. LMCache), you might share a cache across processes for repeated prompts.
- **Prompt Caching (Prefill Caching):** If many requests share a common prefix (e.g. a system prompt), you can *pre-compute* its KV once and reuse it. In HF Transformers, you can build a `StaticCache` with that prefix and then reuse it (see the prefix code above). This is like memoizing the common part of all queries.
- **Cache Reuse in Dialog:** In chat apps, you normally carry the same cache forward. In HF's generator loop, you pass the previous `past_key_values` into the next call, so the model "remembers" the dialogue history without re-processing it. You essentially have an *infinite growing context* (bounded by memory) ²¹ .

- **Sliding-Window Models:** Many modern LLMs (Mistral, Gemma, Llama4) use **sliding-window attention** by design. In those models, the cache naturally caps at the window size: any older tokens beyond the window simply drop out of attention ²². Transformers’ docs note that for sliding/chunked layers, the cache never grows beyond the layer’s window length. In other words, you automatically evict old entries as new ones come.
- **Sink Cache (Attention Sinks):** A hybrid strategy is to keep **only a few key “sink” tokens** (e.g. the very first prompt tokens) permanently, and drop everything else as you go ²³. Hugging Face’s *SinkCache* (in `transformers>=4.53`) implements this: it keeps the first N tokens in cache, discarding older ones beyond a sliding window. This allows “infinite” generation by sacrificing some early context. It’s a middle ground between full recall and fixed windowing ²³.

In practice, deployments often mix static pre-caching (for repeated prefixes) with dynamic continuation (for dialog), plus memory tactics (offload, quantize, evict) to fit hardware. Monitoring cache growth is important: KV cache size = $O(\text{batch_size} \times \text{seq_len} \times \text{model_dim})$, so very long chats or batches can quickly exhaust RAM. Many systems enforce a maximum context or periodically trim history.

3.3 Multi-Turn Dialogues and Long Contexts

KV caching is **essential** for chatbots and multi-turn applications. The trick is simply **to carry the cache forward** across turns. Concretely, after generating a reply, the `past_key_values` tensor holds the entire conversation’s K/V. When the user asks the next question, we feed that `past_key_values` into the next `generate()` call. The model then attends to *all previous turns* efficiently via the cache, without re-encoding them ²¹. Pseudocode:

```
past_cache = DynamicCache(model.config)
for turn in conversation:
    inputs = tokenizer(turn, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, past_key_values=past_cache,
                             max_new_tokens=128)
    reply = tokenizer.decode(outputs[0], skip_special_tokens=True)
    conversation.append({"role": "assistant", "content": reply})
```

This loop “remembers” the whole dialogue. HF’s default `generate` will even accept a `past_key_values` to continue generation indefinitely (until memory). In this way, chat models can handle long history smoothly.

Managing very long context: Because KV size grows linearly, truly *infinite* history is impractical. Common strategies include:

- **Trimming:** Cap the conversation length and drop oldest turns if exceeded.
- **Sliding-window attention:** If the model supports it (e.g. local attention), simply let the cache evict old tokens as needed ²².
- **Attention Sinks:** As above, keep only crucial initial tokens (like system prompt instructions) and discard older conversational tokens ²³. This means the assistant “forgets” earlier user utterances but retains core guidelines.
- **Selective Compression:** Research (e.g. H2O, Scissorhands, StreamingLLM) suggests we can prune low-

impact tokens from cache to cut it dramatically (even ~80% smaller) with minimal loss ¹². These are advanced techniques not yet standard in frameworks, but the idea is that not all words need equal memory.

In summary, multi-turn caching means the model never re-reads history. For truly massive contexts, one must either rely on model features (sliding window) or smart pruning. Otherwise, KV cache size will eventually hit hardware limits, forcing truncation or more aggressive caching policies.

Chapter 4: Advanced Topics

4.1 Quantization and Cache Compression

To further reduce memory, several techniques exist:

- **Quantized KV Cache:** Many frameworks now support low-precision caches. For example, vLLM and TensorRT-LLM typically use **8-bit** (FP8/E4M3) caches, and NVIDIA's newer releases support INT8 as well. Hugging Face's `QuantizedCache` (e.g. via BitsAndBytes) can store K/V at 4-bit or 2-bit, though extremely low-bit often hurts output quality ¹². Studies show 16→4 bit gives 4–8× smaller cache but with noticeable accuracy loss ¹². Hence, 8-bit is a common sweet spot: it halves memory (vs 16-bit) with negligible drop. In code you'd enable it via `cache_implementation="quantized"` and specify the backend. Note that FlashAttention may not work with quantized cache, so some systems switch to a different attention kernel when caching is quantized.
- **Multi-Query / Grouped-Query Attention (MQA/GQA):** These are *architectural* approaches. In MQA, all attention heads share one K/V pair per layer instead of each head having its own ²⁴. This means the KV cache tensor is smaller by roughly the number of heads. For example, if a model has 16 heads but you use MQA (1 shared key per layer), the cache is ~16× smaller. The trade-off is that the model can be slightly less expressive, but many recent LLMs (e.g. Mistral-7B) use MQA for efficiency. NVIDIA explains that with MQA, the *compute* per token is the same, but *memory reads* are far fewer, improving throughput when memory is the bottleneck ²⁴. GQA (grouped-query attention) is a middle ground, sharing keys among small groups of heads. These methods effectively shift the balance toward compute vs memory.
- **Cache Pruning/Compression Algorithms:** Beyond quantization, cutting tokens from cache can save memory. Techniques like **StreamingLLM** (always keep first N + recent M tokens), **H2O**, and **Scissorhands** evaluate attention scores and drop "unimportant" tokens from the cache to fit a budget. In experiments, such algorithms can reduce KV cache size by ~50–80% with minimal impact on generation ¹². However, they require custom logic (not just a flag) and are typically research prototypes. They represent an emerging approach: *throw away some context that the model won't need*, trading a little accuracy for huge memory savings.
- **Trade-offs:** Each method involves trade-offs. Quantizing adds dequantization overhead; multi-query attention can slightly change model behavior; compression might drop subtle context. In practice, the best path depends on constraints: if GPU RAM is ample, simply use full-precision cache for maximum speed. If memory is limited, try 8-bit or offloading first. If you *really* need more, consider MQA/GQA or even cache pruning.

4.2 Fine-Tuning and Checkpointing

During **training** (pretraining or fine-tuning), KV caching is usually *not* employed: training feeds full contexts through in parallel, without autoregressive loops, so there is no incremental “past_key_values” concept. All frameworks process full sequences, and gradients flow through fresh computations. We found no standard use of KV cache across gradient updates. In short, KV caching is primarily an **inference-time** optimization.

One might wonder about **checkpointing/resuming** a long generation. Currently, there is no built-in “save and reload KV cache to disk” feature in mainstream libraries. Technically, the `past_key_values` are just tensors, so you *could* `torch.save()` them. But plugging them back in to continue generation seamlessly is not officially supported or documented. In practice, if you need to stop and resume generation far out, you often just keep the process alive or recompute the cache by re-running the prefix. The exception is using a SinkCache or a sliding window, which inherently “resets” old keys.

Summary of Trade-offs: Overall, KV caching exemplifies classic memory-vs-compute tradeoffs. A full on-device cache uses more RAM but avoids recomputation. A small cache (or offloaded cache) saves memory but may re-run some key/V computations each token or incur transfer latency. The right balance depends on your hardware and workload. As one NVIDIA blog concludes: *if GPU RAM is ample, use a full-float cache for speed; if memory is tight, use offloading or 8-bit caching* ⁶ ¹⁰. In clustered serving, sharing and eviction policies (priority-based reuse) can amortize the cost across requests ²⁰.

Chapter 5: Code Examples and Diagrams

Below are some concrete code snippets illustrating common patterns. These are intended to be illustrative, not fully polished production scripts.

Example 1: Basic generation with KV cache (HF Transformers)

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2").to("cuda")

prompt = "Once upon a time"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

# By default use_cache=True, so model.generate() will cache past_key_values
outputs = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

This runs the autoregressive loop with caching under the hood. Each new token’s K/V is saved automatically. If you call `generate` again on the same model with a `past_key_values` argument, it will reuse the cached history.

Example 2: Prefix Caching

```
from transformers import AutoModelForCausalLM, AutoTokenizer, StaticCache

tokenizer = AutoTokenizer.from_pretrained("gpt2-medium")
model = AutoModelForCausalLM.from_pretrained("gpt2-medium").to("cuda")

prefix = "You are a helpful assistant. "
cache = StaticCache(config=model.config, max_cache_len=50)

inputs = tokenizer(prefix, return_tensors="pt").to(model.device)
with torch.no_grad():
    cache = model(**inputs, past_key_values=cache).past_key_values

questions = ["How are you?", "Tell me a joke about cats."]
for q in questions:
    text = prefix + q
    inputs2 = tokenizer(text, return_tensors="pt").to(model.device)
    # Copy the cached prefix for this query
    prefix_cache = copy.deepcopy(cache)
    outputs = model.generate(**inputs2, past_key_values=prefix_cache,
max_new_tokens=30)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
    print(answer)
```

Here we first run the model on a common `prefix` to get its `past_key_values`. We then reuse that cache (by copying) for different continuations. This means the prefix is only computed once, saving time.

Example 3: Chat with DynamicCache

```
from transformers import AutoModelForCausalLM, AutoTokenizer, DynamicCache

model_id = "meta-llama/Llama-2-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id).to("cuda")

conversation = [
    {"role": "user", "content": "Hello, who are you?"},
    {"role": "assistant", "content": "I am an AI assistant."}
]

cache = DynamicCache(config=model.config)
for turn in conversation:
    text = tokenizer.apply_chat_template([turn], add_generation_prompt=True)
    inputs = tokenizer(text, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=50, past_key_values=cache)
```

```
reply = tokenizer.decode(outputs[0, -50:], skip_special_tokens=True)
print(reply)
```

This loop shows a simple multi-turn dialogue. The `DynamicCache` object accumulates all KV states. Each new `generate()` call with `past_key_values=cache` lets the model attend to the full history. The model “remembers” prior turns via the cache without re-processing them.

Figure (placeholder):

This chapter is code-focused, but one could also include diagrams such as memory layouts or attention flow charts. For example, a diagram contrasting contiguous vs paged KV storage would illustrate vLLM’s approach. (Due to format constraints, we have shown one illustrative figure for the basic KV workflow earlier.)

Chapter 6: Research Landscape and Best Practices

Academic Context: The importance of KV cache for efficient inference has only recently been recognized. Papers like *PagedAttention* (vLLM) at SOSP’23 formalized the memory paging idea ⁷. Other works (e.g. *Attention Sinks* by Steam et al., *Scissorhands*, *H2O*) explore pruning old tokens. Benchmarks and blogs (Neptune.ai, NVIDIA) analyze cache sizes and trade-offs. The NVIDIA developer blog “Mastering LLM Techniques” summarizes these optimizations at a high level, noting strategies like MQA, FlashAttention, and PagedAttention ¹⁹ ²⁴.

Best Practices Summary:

- **Always use KV caching for generation:** It’s a free speedup. For example, simply enabling `use_cache=True` gave a Hugging Face example a ~5× speed boost ⁵.
- **Profile memory vs compute:** If GPU RAM is ample, use a full-precision cache on GPU. If memory is the limit, enable offloading or 8-bit cache quantization ⁶ ¹².
- **Use optimized kernels:** Deploy FlashAttention or Triton kernels to accelerate attention math ¹⁹.
- **Leverage batching and parallelism:** Batch requests when possible, and exploit model parallelism (tensor/pipeline) to fit large models or larger total batch sizes ⁶.
- **Enable advanced cache features:** If using libraries like TensorRT-LLM, tune cache policies (paging, priorities) to your workload. For chatbots, carry caches forward turn by turn. For systems with repeated prompts, prefill or share caches.
- **Cap or trim long context:** For very long sequences, use sliding or sink strategies to bound cache size ²² ²³. In practice, most applications have a maximum context size anyway.
- **Monitor and adapt:** Track KV cache growth in your system. Consider logging cache usage and eviction events (some frameworks provide hooks). Adjust context windows or offload settings if you see OOMs.

The field is evolving: future work includes even more sophisticated cache compression and distributed caching (e.g. LMCache/CDN for shared prompts), better integration with training, and hardware support (like NVIDIA’s NVFP4 format). But the core message is clear: **KV caching is fundamental for efficient Transformer inference**. Understanding and tuning it – from code patterns to hardware kernels – can yield order-of-magnitude improvements in performance ⁵ ¹⁵.

Sources: We have drawn on Hugging Face and NVIDIA documentation and blogs ¹ ² ¹⁹ ²⁰, academic papers on caching and paging ⁷, and performance studies ⁵ ¹². These sources provide the quantitative and conceptual underpinnings for the above discussion.

¹ ³ ⁴ ⁵ KV Caching Explained: Optimizing Transformer Inference Efficiency

<https://huggingface.co/blog/not-lain/kv-caching>

² ¹⁰ ¹¹ ²² KV cache strategies

https://huggingface.co/docs/transformers/en/kv_cache

⁶ ⁸ ¹⁹ ²⁴ Mastering LLM Techniques: Inference Optimization | NVIDIA Technical Blog

<https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/>

⁷ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ Part 2 — Memory Is the Real Bottleneck: How Paged Attention Powers the vLLM Inference Engine | Data Science Dojo

<https://datasciencedojo.com/blog/understanding-paged-attention/>

⁹ ¹⁸ ²¹ Chapter 1_ Foundations of KV Caching.pdf

file:///file_0000000051387209be291b35c838066e

¹² [vLLM vs TensorRT-LLM] #8. KV Cache Quantization - SqueezeBits

<https://blog.squeezebits.com/vllm-vs-tensorrtllm-8-kv-cache-quantization-35079>

²⁰ Introducing New KV Cache Reuse Optimizations in NVIDIA TensorRT-LLM | NVIDIA Technical Blog

<https://developer.nvidia.com/blog/introducing-new-kv-cache-reuse-optimizations-in-nvidia-tensorrt-llm/>

²³ transformers-community/sink_cache · Hugging Face

https://huggingface.co/transformers-community/sink_cache